

BÀI THỰC HÀNH

LAB 06

Điều khiển độ sáng LED bằng biến trở

Kết hợp Analog Input và PWM Output với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	Từ Analog Input tới PWM Output	2
2.2	Ánh xạ miền giá trị	2
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Ánh xạ chân sử dụng	3
3.3	Kết nối biến trở	4
4	EduFramework API	4
5	Bài thực hành	4
5.1	Yêu cầu	4
5.2	Chương trình	5
5.3	Kiểm chứng	5
6	Mở rộng	5
7	Tài liệu tham khảo	6

1 Giới thiệu

1.1 Tổng quan bài lab

Lab 04 đã sử dụng ADC để đọc tín hiệu analog, trong khi Lab 05 sử dụng PWM để điều khiển độ sáng LED. Bài lab này kết hợp hai kiến thức đó thành một luồng xử lý hoàn chỉnh: đọc vị trí biến trở, chuyển đổi giá trị ADC sang miền PWM và sử dụng kết quả để điều khiển trực tiếp độ sáng của LED.

Biến trở tạo ra một điện áp thay đổi tại chân giữa khi vị trí núm xoay thay đổi. Điện áp này được đưa vào ADC0_SE12, sau đó `analogRead()` trả về giá trị ADC 12-bit. Kết quả được ánh xạ từ miền 0 đến 4095 sang miền 0 đến 255 trước khi truyền cho `analogWrite()`.

Bài lab không giới thiệu API analog mới. Trọng tâm là cách kết hợp Analog Input, xử lý dữ liệu và PWM Output thành một ứng dụng tương tác đơn giản.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả luồng xử lý từ tín hiệu analog đầu vào tới PWM đầu ra.
2. Thực hiện ánh xạ tuyến tính giá trị ADC 12-bit từ 0..4095 sang miền PWM 0..255.
3. Kết hợp `analogRead()` và `analogWrite()` trong cùng một ứng dụng.
4. Xây dựng và kiểm chứng ứng dụng điều khiển độ sáng LED bằng biến trở.

2 Kiến thức nền

2.1 Từ Analog Input tới PWM Output

Một ADC chuyển điện áp analog thành một giá trị số để chương trình có thể xử lý. Với độ phân giải 12-bit, có 4096 mã số khác nhau và miền kết quả có thể biểu diễn từ 0 đến 4095 [1].

Ở phía đầu ra, bài lab sử dụng `analogWrite()` với miền điều khiển từ 0 đến 255. Vì miền giá trị ADC và miền PWM khác nhau, kết quả đọc từ ADC không được truyền trực tiếp sang PWM mà cần được chuyển đổi về miền phù hợp.

Luồng dữ liệu của bài lab có thể biểu diễn như sau:

Biến trở → ADC0_SE12 → `analogRead()` → Scaling → `analogWrite()` → LED_RED

Cách tổ chức này tách ứng dụng thành ba bước rõ ràng: thu nhận dữ liệu, xử lý dữ liệu và điều khiển đầu ra.

2.2 Ánh xạ miền giá trị

Bài thực hành cần chuyển một giá trị ADC trong miền 0..4095 thành một giá trị PWM trong miền 0..255. Khi hai miền cùng bắt đầu từ 0, phép ánh xạ tuyến tính có thể viết:

$$PWM = ADC \times \left(\frac{255}{4095} \right)$$

Trong chương trình C, phép tính được thực hiện bằng:

```
pwmValue = (adcValue * 255) / 4095;
```

Cách ánh xạ giá trị analog đầu vào sang miền PWM đầu ra cũng được sử dụng trong ví dụ Analog In, Out Serial của Arduino, trong đó kết quả Analog Input được ánh xạ sang miền 0..255 trước khi điều khiển PWM [3].

Giá trị ADC	Giá trị PWM	Mức điều khiển
0	0	LED tắt.
1024	63	Mức PWM thấp.
2048	127	Xấp xỉ giữa miền điều khiển.
3072	191	Mức PWM cao.
4095	255	Độ sáng tối đa.

Bảng 6.2.1. Ví dụ ánh xạ giá trị ADC 12-bit sang miền PWM 0 đến 255

Khi giá trị ADC tăng, giá trị PWM sau phép ánh xạ cũng tăng. Vì vậy, với cách kết nối của bài thực hành, đưa chân giữa của biến trở về gần 3V3 làm giá trị ADC tăng và LED sáng hơn; đưa về gần GND làm giá trị ADC giảm và LED tối hơn.

Ghi chú

Phép ánh xạ chỉ chuyển đổi giữa hai miền số. Nội dung về quá trình chuyển đổi ADC và nguyên lý PWM đã được trình bày trong Lab 04 và Lab 05 nên không được lặp lại trong bài lab này.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng một biến trở làm Analog Input và LED đỏ tích hợp trên MaaZEDU Development Board làm PWM Output.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
Biến trở	Tạo điện áp thay đổi theo vị trí núm xoay.
Breadboard	Sử dụng để bố trí biến trở và các kết nối ngoài.
Jumper wires	Kết nối 3V3, GND và tín hiệu ADC giữa biến trở và board.
USB Cable	Kết nối board với máy tính để cấp nguồn và nạp chương trình.

Bảng 6.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân sử dụng

Bài thực hành sử dụng ADC0_SE12 làm Analog Input và LED_RED làm PWM Output. Trên S32K144, Channel 12 của ADC0 được ánh xạ tới PTC14; NXP cũng sử dụng Channel 12 làm đầu vào từ biến trở trong ví dụ ADC của S32K144 [2].

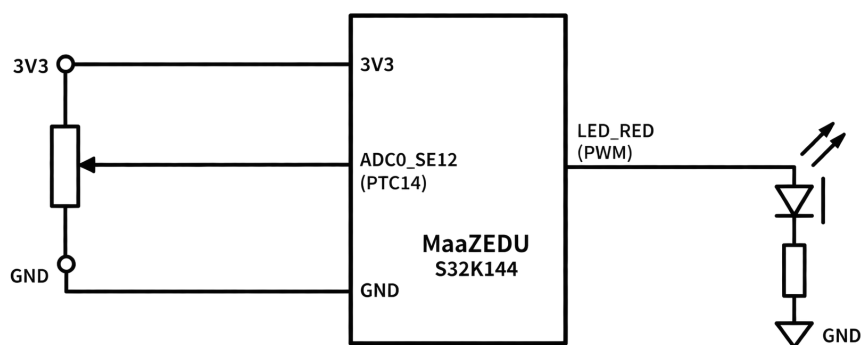
Thành phần	Logical Pin	MCU Pin	Chức năng
Chân giữa biến trở	ADC0_SE12	PTC14	Analog Input / ADC0 Channel 12
LED đỏ tích hợp	LED_RED	PTD15	PWM Output

Bảng 6.3.2. Ảnh xạ chân sử dụng trong bài thực hành

3.3 Kết nối biến trở

Hai chân ngoài của biến trở được nối lần lượt với 3V3 và GND. Chân giữa (wiper) được nối tới ADC0_SE12 (PTC14). Khi xoay biến trở, điện áp tại chân giữa thay đổi trong khoảng giữa hai mức nguồn và tạo tín hiệu analog cho ADC.

LED_RED là LED tích hợp trên board nên không cần lắp thêm LED hoặc điện trở ngoài.



Hình 6.3.1. Sơ đồ kết nối biến trở với Analog Input trong Lab 06

4 EduFramework API

Lab 06 không giới thiệu API mới. Chương trình kết hợp hai API đã sử dụng ở các bài trước: `analogRead()` để lấy giá trị ADC và `analogWrite()` để thiết lập mức PWM [4].

Luồng sử dụng trong ứng dụng:

`analogRead(ADC0_SE12) → Scaling 0..4095 → 0..255 → analogWrite(LED_RED, pwmValue)`

`pinMode()` tiếp tục được sử dụng để cấu hình LED_RED ở chế độ OUTPUT như trong các bài lab trước. Chi tiết cú pháp và tham số của các API này không được lặp lại trong bài lab.

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình sử dụng biến trở để điều khiển trực tiếp độ sáng LED_RED.

Chương trình cần đáp ứng các yêu cầu sau:

- Đọc giá trị biến trở thông qua ADC0_SE12.
- Ánh xạ giá trị ADC từ miền 0..4095 sang miền PWM 0..255.
- Sử dụng giá trị sau ánh xạ để điều khiển LED_RED.
- Cập nhật liên tục để LED phản hồi theo vị trí hiện tại của biến trở.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```
#include "Arduino.h"

int main(void)
{
    int adcValue = 0;
    int pwmValue = 0;

    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        adcValue = analogRead(ADC0_SE12);

        pwmValue = (adcValue * 255) / 4095;

        analogWrite(LED_RED, pwmValue);
    }

    return 0;
}
```

Mã nguồn 6.5.1. Chương trình điều khiển độ sáng LED bằng biến trở

`setup()` khởi tạo các thành phần nền tảng của EduFramework và `pinMode(LED_RED, OUTPUT)` cấu hình LED đỏ làm đầu ra.

Trong vòng lặp chính, `analogRead(ADC0_SE12)` đọc điện áp tại chân giữa của biến trở và trả về giá trị ADC. Dòng:

```
pwmValue = (adcValue * 255) / 4095;
```

thực hiện phép ánh xạ từ miền ADC 12-bit sang miền PWM. Giá trị `pwmValue` sau đó được truyền trực tiếp cho `analogWrite()`, vì vậy mỗi thay đổi của biến trở được phản ánh thành thay đổi tương ứng ở độ sáng LED.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board. Xoay biến trở từ vị trí gần `GND` về phía `3V3`, sau đó xoay theo chiều ngược lại và quan sát `LED_RED`.

KẾT QUẢ MONG ĐỢI

Độ sáng của `LED_RED` thay đổi theo vị trí biến trở. Khi điện áp tại chân giữa tăng, giá trị ADC và giá trị PWM tăng nên LED sáng hơn. Khi điện áp tại chân giữa giảm, LED tối dần và có thể trở về trạng thái tắt ở đầu thấp của miền điều khiển.

6 Mở rộng

Bài thực hành chính ưu tiên phản hồi trực tiếp giữa biến trở và LED nên không hiển thị dữ liệu trong vòng lặp. Để quan sát rõ quá trình ánh xạ, có thể sử dụng Serial Monitor đã học ở Lab 03 để hiển thị đồng thời giá trị

ADC đầu vào và giá trị PWM sau xử lý. Việc hiển thị cả giá trị đầu vào và đầu ra cũng được sử dụng trong ví dụ Analog In, Out Serial của Arduino [3] .

Dữ liệu có thể được trình bày theo dạng:

```
ADC: 512 | PWM: 31
ADC: 2048 | PWM: 127
ADC: 3584 | PWM: 223
```

Các số trên chỉ minh họa định dạng hiển thị; giá trị thực tế phụ thuộc vào vị trí biến trở và tín hiệu ADC tại thời điểm đọc.

Ghi chú

Nếu dữ liệu được gửi tới Serial Monitor trong mọi vòng lặp, tốc độ hiển thị có thể quá nhanh để quan sát. Khi thực hiện phần mở rộng, có thể chọn một khoảng thời gian cập nhật phù hợp cho mục đích giám sát.

Mở rộng: Bổ sung Serial Monitor với baud rate 9600 và hiển thị adcValue cùng pwmValue trên cùng một dòng. Xoay biến trở qua nhiều vị trí và kiểm tra rằng giá trị PWM thay đổi từ gần 0 tới gần 255 theo sự thay đổi của kết quả ADC.

7 Tài liệu tham khảo

- [1] The University of Texas at Austin, *Chapter 7: ADC, Data Acquisition, and Control* , Embedded Systems.
- [2] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.
- [3] Arduino, *Analog In, Out Serial* , Built-in Examples.
- [4] EduFramework, *Analog API* , EduFramework Source Code.